



2강. 로봇 소프트웨어 스택과 실시간성



학습 목표

- 로봇 소프트웨어 스택의 인지-판단-제어 계층 구조와 각 계층의 역할을 설명할 수 있습니다.
- 세 계층 사이의 데이터 흐름과, 계층마다 갱신 주기가 다른 멀티레이트 (multi-rate) 구조를 이해합니다.
- 실시간성 관점에서 주기(period)·지연(latency)·지터(jitter)를 정확히 구분하고 설명할 수 있습니다.
- Hard/Firm/Soft 실시간의 차이를 이해하고, 로봇의 작업이 어디에 속하는지 판단할 수 있습니다.
- 지터가 발생하는 원인과 이를 줄이는 접근(RTOS 등)을 개념 수준에서 설명할 수 있습니다.



목차

들어가며

1. 로봇 소프트웨어 스택의 세 계층

인지 (Perception)

판단 (Planning)

제어 (Control)

2. 데이터는 어떻게 흐를까요? — 멀티레이트 구조

3. 실시간성 ① — 주기·지연·지터

왜 지터가 제어를 망가뜨릴까요?

4. 실시간성 ② — Hard / Firm / Soft 실시간

5. 지터는 어디서 생기고, 어떻게 줄일까요?

6. 실생활 시나리오로 통합하기

미니 퀴즈

실습

정리

들어가며

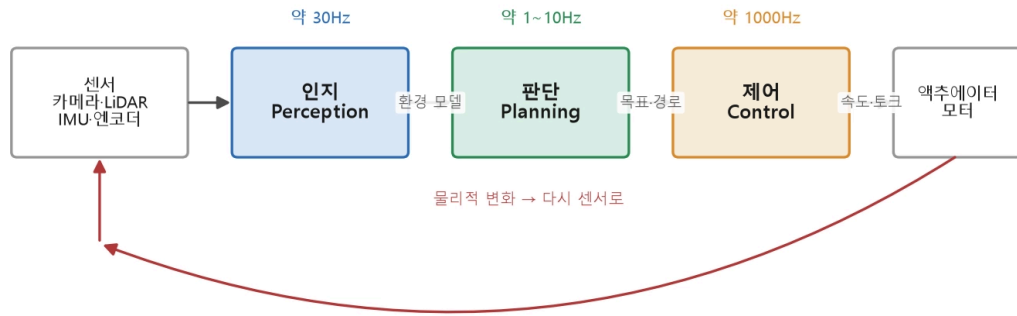
사람이 뜨거운 주전자를 잡았을 때를 떠올려 보겠습니다. **피부가 뜨거움을 느끼고(인지) → 위험하다고 판단하고(판단) → 손을 땡니다(행동)**. 이 과정은 눈 깜짝할 사이에 일어나지만, 분명히 순서가 있는 하나의 흐름입니다. 흥미로운 점은, 실제로 이 반사 행동은 뇌까지 가지 않고 **척수에서 바로 처리**된다는 것입니다. "빠른 판단은 가까운 곳에서, 복잡한 판단은 먼 곳(뇌)에서" — 1강에서 본 연산 부담의 원리가 우리 몸에도 있는 셈입니다.

로봇도 똑같습니다. 로봇의 소프트웨어는 **세상을 읽고, 무엇을 할지 정하고, 몸을 움직이는 세 계층**으로 구성됩니다. 이 구조를 이해하면 아무리 복잡한 로봇 코드도 "지금 이 코드가 어느 계층의 일인지" 파악할 수 있습니다. 그리고 이 계층들이 **제때** 돌아가게 만드는 것이 실시간성입니다.

1. 로봇 소프트웨어 스택의 세 계층

로봇 소프트웨어는 크게 **인지(Perception) – 판단(Planning/Decision) – 제어(Control)** 세 계층으로 나뉩니다.

계층마다 갱신 속도가 다릅니다



images/02_인지판단제어_파이프라인.png — 로봇 소프트웨어 스택: 인지 → 판단 → 제어

💡 **해석:** 데이터는 왼쪽(센서)에서 오른쪽(모터)으로 흐릅니다. 인지는 센서 데이터를 **의미 있는 환경 정보**로 바꾸고, 판단은 그 정보로 **무엇을 할지** 결정하며, 제어는 결정을 **정밀한 모터 명령**으로 변환합니다. 로봇이 움직이면 환경이 바뀌어 다시 센서로 들어옵니다. 이 **폐루프(닫힌 고리)**가 로봇이 켜져 있는 내내 돌아 갑니다. 각 계층 위의 주파수(30Hz·1~10Hz·1000Hz)에 주목하세요 — 계층마다 도는 속도가 다릅니다.

각 계층을 조금 더 구체적으로 살펴보겠습니다.

인지 (Perception)

센서 원시 데이터를 처리해 "지금 상황"을 파악합니다.

- **입력:** 카메라 영상, LiDAR 점군, IMU 가속도·각속도, 엔코더 회전량 등
- **출력:** "장애물이 전방 1.2m에 있다", "내 위치는 지도상 (3.4, 7.1)이다", "로봇이 5° 기울었다" 같은 **상태 추정치**
- **대표 알고리즘:** 객체 검출, SLAM(동시적 위치추정·지도작성), 센서 융합(Kalman 필터 계열)

여기서 중요한 개념이 **상태 추정(state estimation)**입니다. 센서는 노이즈가 섞인 단편적 정보만 주기 때문에, 인지 계층은 여러 센서를 융합해 "**가장 그럴듯한 현재 상태**"를 계산합니다. 예를 들어 IMU는 빠르지만 오차가 누적되고, GPS는 느리지만 절대 위치를 줍니다. 이 둘을 융합하면 빠르고 정확한 위치 추정이 됩니다.

판단 (Planning)

인지 결과와 목표를 바탕으로 "무엇을 할지" 결정합니다.

- **입력:** 환경 모델(장애물 지도, 내 위치), 상위 목표("현관까지 이동")
- **출력:** 경로(waypoint 목록), 행동 결정("왼쪽으로 회피"), 속도 프로파일

- **대표 알고리즘:** 전역 경로 계획(A*, Dijkstra), 지역 경로 계획(DWA), 행동 트리(Behavior Tree)

판단은 보통 **전역(global)** 과 **지역(local)** 로 나뉩니다. 전역 계획은 "집 전체 지도에서 현관까지 가는 길"을 짜고(느려도 됨), 지역 계획은 "지금 눈앞의 장애물을 어떻게 피할지"를 짧은 주기로 갱신합니다.

제어 (Control)

판단이 정한 목표를 "실제 모터 명령"으로 바꿉니다.

- **입력:** 목표 속도·위치("좌측 바퀴 0.5m/s")와 현재 측정값(엔코더)
- **출력:** 모터 전압·전류·PWM 듀티
- **대표 알고리즘:** PID 제어(모듈 ③에서 상세히 다룹니다), 모델 기반 제어

제어 계층의 특징은 **오차를 끊임없이 보정**한다는 점입니다. "0.5m/s로 돌아라"라고 명령해도 마찰·경사·배터리 전압에 따라 실제 속도는 달라집니다. 제어기는 목표와 측정의 **차이(오차)** 를 매 주기 계산해 출력을 조정합니다.

2. 데이터는 어떻게 흐를까요? — 멀티레이트 구조

세 계층은 독립적으로 존재하지 않고 끊임없이 데이터를 주고받습니다. 중요한 점은 계층마다 **데이터가 갱신되는 속도가 다르다**는 것입니다. 이를 **멀티레이트(multi-rate)** 구조라 합니다.

계층	전형적 주기	주기를 결정하는 요인
제어	100Hz~10kHz	모터·기구의 동역학 (빠를수록 정밀·안정)
인지	10~60Hz	센서 프레임률 (카메라 30fps, LiDAR 10~20Hz)
판단(지역)	5~20Hz	환경 변화 속도
판단(전역)	0.1~1Hz	목표·지도의 변화 빈도

그렇다면 빠른 제어 루프(1kHz)가 느린 인지 결과(30Hz)를 어떻게 쓸까요? **기다리지 않습니다.** 제어 루프는 "인지가 마지막으로 준 최신 값"을 들고 자기 속도로 계속 돕니다. 인지가 새 결과를 내놓으면 그때 갈아끼웁니다.

- ✓ **핵심 원칙: 빠른 루프는 느린 루프를 기다리지 않는다.** 각 계층은 자기 주기로 돌고, 계층 간에는 "가장 최신 값"을 비동기로 공유합니다. 이것이 ROS2의 Topic 통신(모듈 ②)이 비동기 발행-구독 구조인 이유이기도 합니다.

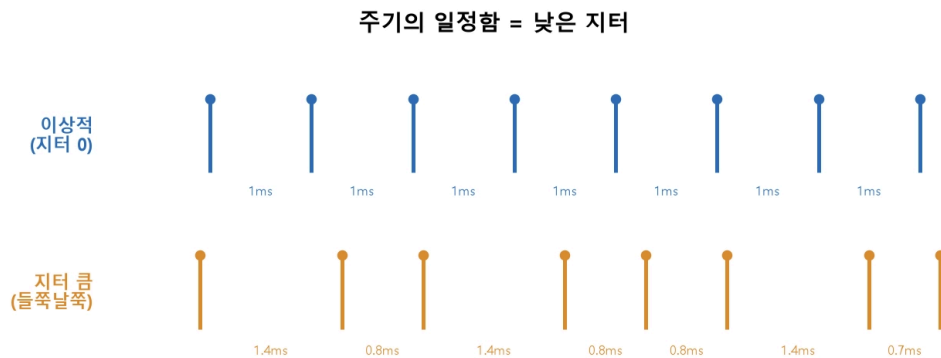
만약 이 원칙을 어기고 제어 루프가 인지를 기다리면 어떻게 될까요? 제어 주기가 1ms → 33ms로 무너지고, 모터 제어가 거칠어지며, 최악의 경우 시스템이 불안정해집니다. 이런 설계 실수를 **우선순위 역전과 블로킹**이라고 부르며, 로봇 소프트웨어에서 가장 흔한 성능 버그 중 하나입니다.

3. 실시간성 ① — 주기·지연·지터

로봇 제어가 "제때" 이루어지는지를 나타내는 세 가지 개념이 있습니다. 서로 헷갈리기 쉬우니 정확히 구분하겠습니다.

- **주기(Period)**: 작업이 **얼마나 자주** 반복되는가. 예) 1ms마다 모터 제어 → 주기 1ms(=1000Hz). 주파수(frequency)는 주기의 역수입니다.
- **지연(Latency)**: 입력이 들어와서 출력이 나올 때까지 **걸리는 시간**. 예) 센서값을 읽고 모터 명령이 나오기까지 0.3ms.
- **지터(Jitter)**: 주기(또는 지연)가 **얼마나 들쭉날쭉한가**. 매번 정확히 1ms면 지터 0, 어떤 때는 0.9ms 어떤 때는 1.3ms면 지터가 큼니다.

세 개념은 독립적입니다. 주기가 빨라도 지연이 길 수 있고(파이프라인이 깊은 경우), 평균 주기가 정확해도 지터가 클 수 있습니다.



images/02_지터_타임라인.png — 주기의 일정함과 지터

💡 **해석**: 위는 정확히 1ms 간격으로 실행되는 이상적인 경우(지터 0), 아래는 간격이 0.7~1.4ms로 흔들리는 경우입니다. **평균 주기는 같아도, 지터가 크면 제어가 불안정해집니다.** 로봇 제어에서는 평균 속도만큼이나 **일정함(낮은 지터)**이 중요합니다.

왜 지터가 제어를 망가뜨릴까요?

PID 같은 디지털 제어기는 대부분 "**일정한 주기 Δt 로 실행된다**"는 가정 위에서 설계됩니다. 적분항은 **오차 $\times \Delta t$** 를 누적하고, 미분항은 **오차 변화 $\div \Delta t$** 를 계산합니다. 실제 Δt 가 설계값과 다르게 출력되면:

- 적분·미분 계산이 매 주기 틀어지고,
- 제어 출력에 노이즈가 실리며,

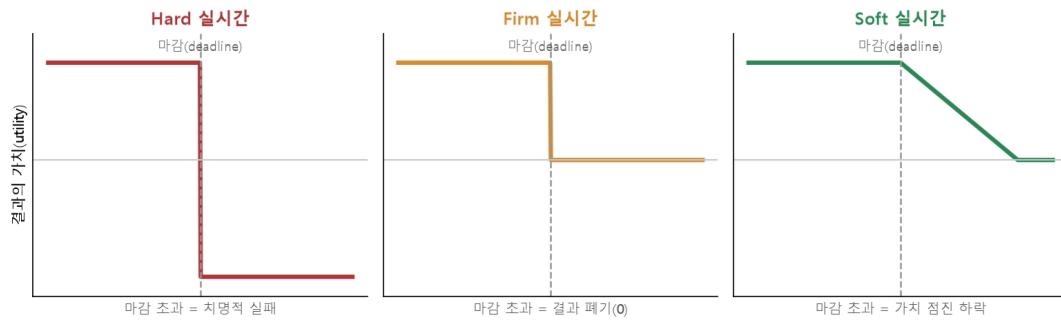
- 심하면 시스템이 진동하거나 발산합니다.

즉 지터는 단순한 "성능 저하"가 아니라 **제어 이론의 전제를 깨뜨리는 문제**입니다. 모듈 ③에서 PID를 구현할 때 이 감각이 다시 등장합니다.

4. 실시간성 ② — Hard / Firm / Soft 실시간

"실시간(real-time)"은 "빠르다"는 뜻이 아니라 **"정해진 마감(deadline) 안에 끝나는 것을 보장한다"** 는 뜻입니다. 마감을 어겼을 때의 결과에 따라 세 등급으로 나눕니다.

실시간성의 분류: 마감을 넘겼을 때의 결과



images/02_실시간_분류.png — 실시간성의 분류

💡 **해석:** 가로축은 작업 완료 시각, 세로축은 그 결과의 가치입니다.

- **Hard 실시간:** 마감을 넘기는 순간 가치가 음수 — 즉 **치명적 실패**가 됩니다. 예) 모터 전류 제어, 에어백 전개, 로봇 낙하 방지. "늦은 정답"은 정답이 아닙니다.
- **Firm 실시간:** 마감을 넘긴 결과는 **가치가 0이 되어 폐기**되지만, 시스템이 붕괴하지는 않습니다. 예) 화상회의의 영상 프레임 — 늦게 도착한 프레임은 그냥 버립니다.
- **Soft 실시간:** 마감을 넘겨도 가치가 **점진적으로 하락**할 뿐입니다. 예) 로봇 상태 모니터링 대시보드 — 0.5초 늦어도 쓸모는 있습니다.

로봇 시스템에 적용하면 이렇게 나눕니다.

작업	등급	이유
모터 전류/속도 제어 루프	Hard	마감 초과 = 제어 불안정·하드웨어 손상
비상 정지(E-Stop) 처리	Hard	마감 초과 = 안전사고
카메라 프레임 기반 객체 인식	Firm	늦은 프레임 결과는 폐기하고 다음 프레임 처리
지도 갱신·경로 재계획	Soft	조금 늦어도 이전 경로로 주행 가능
로그 업로드·모니터링	Soft	지연이 품질에 거의 영향 없음

이 분류가 중요한 이유는, **등급에 따라 실행 환경이 달라지기 때문**입니다. Hard 실시간 작업은 지터를 보장하는 임베디드(MCU, RTOS)에 두고, Firm/Soft는 리눅스가 도는 Edge 컴퓨터에 둡니다. 1강의 연산 부담과 정확히 맞물리는 구조입니다.

5. 지터는 어디서 생기고, 어떻게 줄일까요?

일반 PC/리눅스에서 1ms 주기 루프를 돌려 보면 주기가 미세하게 출렁입니다. 원인은 여러 겹입니다.

- **OS 스케줄러**: 리눅스는 기본적으로 공평성을 추구하는 범용 스케줄러라, 내 프로세스가 언제 CPU를 받을지 정확히 보장하지 않습니다.
- **인터럽트·다른 프로세스**: 네트워크 패킷 처리, 백그라운드 작업이 끼어듭니다.
- **캐시 미스·메모리 페이징**: 같은 코드도 캐시 상태에 따라 실행 시간이 달라집니다.
- **동적 메모리 할당·가비지 컬렉션**: 할당 시간이 일정하지 않습니다(특히 Python 같은 관리형 언어).

이를 줄이는 대표적 접근이 다음과 같습니다.

- **RTOS(Real-Time OS) 사용**: FreeRTOS, Zephyr 등은 **우선순위 기반 선점형 스케줄링**으로 "가장 급한 작업이 반드시 먼저 실행됨"을 보장합니다. MCU에서 Hard 실시간을 구현하는 표준적 방법입니다(모듈 ③에서 만납니다).
- **리눅스 실시간 패치(PREEMPT_RT)**: 리눅스 커널의 선점 지연을 줄여 준실시간 성능을 냅니다. ROS2 실시간 노드에서 활용됩니다.
- **실시간 코드 작성 수칙**: 제어 루프 안에서 동적 할당·파일 I/O·블로킹 통신을 하지 않기, 실행 시간이 일정한(deterministic) 알고리즘 쓰기.
- ▶ **심화** — 워스트케이스 실행 시간(WCET)과 스케줄링 가능성

6. 실생활 시나리오로 통합하기

배달로봇이 보행자를 만나 멈추는 1초를 계층·실시간 관점으로 분해해 보겠습니다.

1. **인지(Firm, 30Hz)**: 카메라 프레임에서 보행자를 검출합니다. 처리에 25ms가 걸려 다음 프레임 전에 끝났습니다. 만약 40ms가 걸렸다면 그 결과는 버리고 최신 프레임을 처리합니다.
2. **판단(Soft, 10Hz)**: "보행자 전방 1.5m → 감속 후 정지" 경로를 재계획합니다. 한 주기(100ms) 늦어도 이전 계획으로 안전하게 감속 중입니다.
3. **제어(Hard, 1kHz)**: 목표 속도가 0으로 바뀌자, 매 1ms마다 모터 전류를 조정해 부드럽게 정지합니다. 이 루프는 지터 수십 μ s 이내로 MCU에서 둡니다.
4. **안전(Hard, 이벤트 구동)**: 만약 범퍼가 눌리면 위 모든 계층을 건너뛰고 **즉시** 모터 전원을 차단합니다.

한 번의 "멈춤"에도 Hard/Firm/Soft가 공존하며, 각자 알맞은 계층·주기로 협력하는 것을 볼 수 있습니다.

미니 퀴즈



아래 문항을 먼저 스스로 풀어 본 뒤 실습으로 넘어가세요. 답안은 실습 결과물과 함께 제출합니다. (총 10점)

1. (2점) 인지-판단-제어 세 계층의 전형적인 주기를 **빠른 것부터** 나열한 것은?

- ① 인지 > 판단 > 제어
- ② 제어 > 인지 > 판단
- ③ 판단 > 제어 > 인지
- ④ 제어 > 판단 > 인지

2. (2점) **지터(jitter)** 를 한 문장으로 정의하고, 그것이 제어를 망가뜨리는 이유를 쓰세요.

3. (2점) 다음을 Hard / Firm / Soft 실시간으로 분류하세요.

(가) 비상 정지 처리 (나) 카메라 프레임 객체 인식 (다) 로그 업로드

4. (2점) 참/거짓: 리눅스는 범용 OS이므로 어떤 설정으로도 Hard 실시간을 보장할 수 없다.

5. (2점) 지터를 줄이는 방법을 **두 가지** 쓰세요.

실습

1. 여러분이 자주 쓰는 전자기기(스마트폰 게임, 무선 이어폰, 드론 등) 하나를 골라, 그 기기에서 **주기·지연·지터**가 각각 무엇에 해당하는지 한 문장씩 적어보세요.

예) 무선 이어폰 — 주기: 오디오 샘플 재생 간격 / 지연: 영상과 소리가 어긋나는 전송 시간 / 지터: 소리가 끊기거나 튀는 정도

2. 아래 로봇 작업들을 Hard/Firm/Soft로 분류하고 근거를 적어보세요.

- 드론의 자세 안정화 루프
- 로봇청소기의 방 지도 갱신
- 배달로봇의 카메라 기반 신호등 인식
- 로봇팔의 관절 토크 제어
- 클라우드의 주해 루거 언로드